

# Spec-Driven Development

Como estruturar o desenvolvimento com IA para projetos que escalam

Departamento WEB · 2026

## 1. O que é Spec-Driven Development

Spec-Driven Development (SDD) é uma abordagem de desenvolvimento em que uma especificação formal é criada antes de qualquer linha de código. Essa especificação atua como fonte única de verdade - orientando o agente de IA, os testes e as revisões de código.

A diferença central em relação a uma documentação tradicional está na estrutura: a spec SDD é escrita para ser consumida diretamente pela IA, sem ambiguidade e sem floreio.

Formato EARS (Easy Approach to Requirements Syntax) - exemplo prático:

```
WHEN um usuário conclui uma compra com sucesso THE SYSTEM SHALL enviar e-mail de
confirmação para o endereço cadastrado AND incluir no e-mail: número do pedido, itens
comprados e valor total
```

Cada afirmação tem trigger, condição, ação e payload - zero margem de interpretação para o agente ou para o revisor humano.

## 2. SDD vs Vibe Coding

Vibe coding funciona bem nos primeiros dias de qualquer projeto. O problema aparece quando o projeto cresce: a IA não tem memória entre sessões e você passa a gastar energia re-explicando arquitetura em vez de produzir.

| Critério                 | Vibe Coding                            | Spec-Driven Development                   |
|--------------------------|----------------------------------------|-------------------------------------------|
| Ponto de partida         | Intenção descrita em linguagem natural | Spec declarativa sem ambiguidade          |
| Memória entre sessões    | Você re-explica a cada abertura        | Agente lê a spec e retoma o contexto      |
| Escopo dos erros         | Grande e espalhado                     | Pequeno e localizado na task              |
| Revisão de código        | "Esse código está bem escrito?"        | "Esse código implementa a spec?"          |
| Custo inicial            | Baixo - começa rápido                  | Médio - exige escrever a spec antes       |
| Custo em projetos longos | Alto - débito de contexto cresce       | Baixo - contexto fica na spec             |
| Adequação                | Protótipos, scripts descartáveis       | Projetos que vão durar mais de uma sprint |

---

### 3. Quando usar - e quando não usar

#### ☑ Use SDD nestes cenários

- Projeto novo do zero - a spec orienta todas as decisões desde o início, evitando débito de contexto desde a primeira sessão.
- Feature nova complexa - qualquer funcionalidade com múltiplos endpoints, regras de negócio ou integrações externas se beneficia de uma spec clara antes do código.
- Feature com requisitos não-negociáveis - segurança, compliance, SLAs, limites de performance. A spec garante que esses critérios não se percam entre sessões.
- Time com mais de um desenvolvedor - a spec funciona como contrato compartilhado entre front, back e QA, eliminando retrabalho por interpretação divergente.
- Projeto que vai ser mantido por meses - o custo de escrever a spec no início é muito menor do que o custo de re-explicar contexto mês após mês.
- Integração com sistemas legados - documentar o que o novo código deve e não deve fazer protege contra quebras silenciosas.

#### ✗ Não use SDD nestes cenários

- Scripts descartáveis e one-offs - um cron que vai rodar uma vez não precisa de spec. Vibe coding é mais rápido e adequado.
- Prototipagem exploratória - quando o objetivo é aprender se uma ideia funciona, a spec congela decisões que você ainda precisa testar.
- Projetos muito curtos - se cabe em dois ou três dias de trabalho solo, o overhead não compensa.
- Times sem disciplina para manter a spec atualizada - spec desatualizada é pior que vibe coding. Ela cria ilusão de controle. Se a regra não for "spec e código mudam no mesmo PR", não adote SDD.

---

### 4. Estrutura de pastas no projeto

A estrutura abaixo é criada uma única vez na raiz do projeto e versionada junto com o código:

```
meu-projeto/
├── .cursor/
│   └── rules/
│       └── sdd.mdc           ← instruções globais para o agente
├── .specify/
│   ├── specs/              ← O QUÊ cada feature faz
│   ├── plans/              ← EM QUE ORDEM implementar
│   └── tasks/              ← cada UNIDADE atômica de trabalho
└── src/
```

A separação das duas pastas é intencional: `.cursor/` é configuração do editor; `.specify/` é conteúdo do projeto. A pasta `.cursor/rules/` instrui o agente; a pasta `.specify/` alimenta o agente com contexto real.

## 5. Como usar no Cursor - passo a passo

### Passo 1 - Configurar o arquivo de rules

Crie o arquivo `.cursor/rules/sdd.mdc` com as instruções globais. Esse arquivo é lido automaticamente pelo Cursor a cada sessão - é o equivalente ao `CLAUDE.md`.

Exemplo de conteúdo mínimo para `sdd.mdc`:

```
# SDD Rules

Antes de implementar qualquer feature:
1. Leia a spec correspondente em .specify/specs/
2. Leia o plano em .specify/plans/ (se existir)
3. Execute apenas a task indicada em .specify/tasks/
4. Valide o resultado contra a spec ao final

Nunca implemente além do escopo da task atual.
Se a spec for ambígua, pergunte antes de assumir.
```

#### Dica

Caso utilize o Claude ou VSCode, a estrutura `.specify/` fica igual em todos os casos. Só o arquivo de instruções globais muda de nome e lugar dependendo da ferramenta.

| Ferramenta        | Equivalente ao <code>.cursor/rules/sdd.mdc</code>    |
|-------------------|------------------------------------------------------|
| Cursor            | <code>.cursor/rules/sdd.mdc</code>                   |
| Claude Code       | <code>CLAUDE.md</code> na raiz do projeto            |
| VSCode + Copilot  | <code>.github/copilot-instructions.md</code>         |
| VSCode + Cline    | <code>.clinerules</code> na raiz                     |
| VSCode + Continue | Configuração de contexto no <code>config.json</code> |

### Passo 2 - Escrever a spec da feature

Crie um arquivo Markdown em `.specify/specs/` para cada feature. O nome deve ser descritivo e em kebab-case.

Exemplo: `.specify/specs/api-monitoramento-users.md`

```
# Spec: API de Autenticação de Usuários

## O que faz
WHEN um usuário envia e-mail e senha válidos em POST /login
THE SYSTEM SHALL retornar um token de acesso JWT
AND registrar o timestamp do último login

WHEN cliente chama GET /users/{id}
THE SYSTEM SHALL retornar os dados do perfil do usuário
OR retornar 404 se o usuário não existir

WHEN um usuário envia POST /logout
THE SYSTEM SHALL invalidar o token de acesso ativo

## O que NÃO faz
- Não envia e-mails de recuperação de senha (responsabilidade do serviço de notificações)
- Não persiste histórico de sessões anteriores (apenas sessão atual)

## Restrições não-negociáveis
- Todos os endpoints exigem header Authorization válido, exceto POST /login
- Resposta do POST /login deve ser < 300ms no P95
```

### Passo 3 - Gerar o plano

Com a spec aberta no Cursor, acione o Agent mode (Ctrl+I ou Cmd+I) e use o @ para referenciar o arquivo:

```
Com base em @.specify/specs/api-monitoramento-users.md,
gere um plano de implementação ordenado em .specify/plans/api-monitoramento-users.md.
Liste as etapas em ordem de dependência.
```

Revise o plano gerado - corrija a ordem se necessário e remova etapas desnecessárias antes de continuar.

### Passo 4 - Decompor em tasks atômicas

Uma task bem feita tem três propriedades obrigatórias:

- Cabe em uma única sessão sem recarregar contexto
- Verificável de forma binária - passou no teste ou não passou
- Independente, ou com dependência explicitamente declarada

Peça ao Cursor para gerar as tasks:

Com base em `@.specify/plans/api-monitoramento-users.md`,  
decomponha em tasks atômicas em `.specify/tasks/api-monitoramento-users.md`.  
Cada task deve ter: descrição, dependências e critério de aceite.

Exemplo de tasks bem decompostas geradas a partir da spec acima:

```
## Task 1.1 – Entidade User
Descrição: criar entidade com campos id, email, senha_hash, ultimo_login
Dependência: nenhuma
Aceite: entidade persiste no banco, migration aplicada com sucesso

## Task 1.2 – Endpoint POST /login
Descrição: implementar endpoint que valida e-mail e senha e retorna token JWT
Dependência: Task 1.1
Aceite: retorna 200 com token JWT válido ou 401 se credenciais inválidas

## Task 1.3 – Endpoint GET /users/{id}
Descrição: implementar endpoint que retorna dados do perfil do usuário
Dependência: Task 1.1
Aceite: retorna dados do usuário ou 404 se não existir

## Task 1.4 – Endpoint POST /logout
Descrição: implementar endpoint que invalida o token de acesso ativo
Dependência: Task 1.2
Aceite: token invalidado não permite acesso a endpoints protegidos

## Task 1.5 – Middleware de autenticação
Descrição: adicionar middleware que valida header Authorization em todos os endpoints, exceto POST /login
Dependência: Task 1.2, Task 1.3, Task 1.4
Aceite: requisição sem Authorization válido retorna 401
```

## Passo 5 - Implementar task por task

A cada sessão no Cursor, forneça ao agente apenas a task atual e a spec correspondente. Não o projeto inteiro.

Implemente a `@.specify/tasks/api-monitoramento-users.md` Task 1.1.  
Siga as restrições definidas em `@.specify/specs/api-monitoramento-users.md`.  
Ao final, confirme que o critério de aceite foi atendido.

### Atenção

Não peça ao agente para implementar várias tasks de uma vez. O isolamento é o que mantém os erros pequenos e o contexto controlado.

## Passo 6 - Verificar contra a spec

Após cada task, faça a verificação em duas camadas:

- 1. **Testes derivados da spec** - cada WHEN/SHALL vira um caso de teste. Rode a suíte antes de marcar a task como concluída.
- 2. **Revisão humana focada** - a pergunta não é "esse código está bem escrito?", mas sim "esse código faz exatamente o que a spec diz?".

Você pode pedir ao próprio Cursor para fazer uma revisão preliminar:

```
Compare o código implementado com @.specify/specs/api-monitoramento-users.md.  
Liste qualquer ponto onde a implementação diverge da spec.
```

## Passo 7 - Manter a spec atualizada

Regra inegociável: spec e código mudam no mesmo Pull Request.

 **Alerta crítico**

Spec desatualizada é pior que não ter spec. Ela cria ilusão de controle enquanto o time toma decisões baseadas em informação errada. Se a equipe não conseguir manter essa disciplina, não adote SDD - use vibe coding conscientemente.

## 6. Referência rápida

| Artefato              | Finalidade                                         |
|-----------------------|----------------------------------------------------|
| .cursor/rules/sdd.mdc | Instrui o agente com as regras globais do projeto  |
| .specify/specs/*.md   | Define O QUÊ cada feature faz (formato EARS)       |
| .specify/plans/*.md   | Define EM QUE ORDEM implementar                    |
| .specify/tasks/*.md   | Define cada UNIDADE atômica com critério de aceite |

| Camada | Pergunta que responde   | Quem mantém                  |
|--------|-------------------------|------------------------------|
| Spec   | O quê?                  | Dev + PO                     |
| Plan   | Em que ordem?           | Dev líder                    |
| Tasks  | Como dividir?           | Dev responsável pela feature |
| Código | Como implementar?       | Dev + agente de IA           |
| Testes | Foi implementado certo? | Dev + QA                     |

*Vibe coding otimiza o prompt. SDD otimiza a fundação.*